

计算机程序设计基础(2)

---C++程序设计(2)

孙甲松

sunjiasong@tsinghua.edu.cn

电子工程系 信息认知与智能系统研究所

罗姆楼6-104

电话: 13901216180 / 62796193

2020. 2.

1

2. 函数重载

函数重载(function overloading) 是一种数据抽象, 允许函数同名, 函数返回值类型是否相同没有要求, 但要求函数的参数类型和个数不能完全相同。实际工作中许多情况下需要对不同的数据类型作同样的操作。

例如: 求一个数的绝对值, 在C的库函数中有:

- `int abs(int);`
- `long labs(long);`
- `double fabs(double);`
- 分别求`int`, `long`, `double`类型数据的绝对值, 如果用错函数, 往往结果会不正确。为避免出现这个问题, C++允许函数同名, 这种现象称为**函数重载**。
- `int abs(int);`
- `long abs(long);`
- `double abs(double);`

2

2. 函数重载

- 2.1 函数重载实例
- 2.2 函数重载的要求
- 2.3 函数重载与缺省参数

3

2.1 函数重载实例

```
#include <iostream>
using namespace std;
int abs(int i)
{ return i<0?-i:i; }
long abs(long i)
{ return i<0?-i:i; }
double abs(double i)
{ return i<0?-i:i; }
void main( )
{ int a=-10; long b=-65536L; double c=-1.5;
  cout<<abs(a)<<' '<<abs(b)<<' '<<abs(c)<< endl;
}
```

运行结果:
10 65536 1.5
请按任意键继续...

4

2.2 函数重载的要求

函数重载的要求是:

- 函数名相同, 但参数类型和个数不能完全相同。
- 编译系统通过静态绑定(绑定, binding)来确定调用的函数到底是那一个:
 - 当函数参数个数不同时, 由实参数个数确定;
 - 当函数参数个数相同时, 由实参类型确定;
 - 找不到相应个数和类型参数的函数, 编译系统就会报错。
- 所以实参类型是决定性因素, `f(1)`和`f(1.0)`调用的不是同一个函数。C++中调用函数时一定要注意实参类型!

5

有程序:

```
#include <iostream>
using namespace std;
int abs(int i)
{ return i<0?-i:i; }
long abs(long i)
{ return i<0?-i:i; }
float abs(float i)
{ return i<0?-i:i; }
void main( )
{ cout<<abs(-65536);
  cout <<' '<<abs(-65536L);
  cout <<' '<<abs(-1.5)<< endl; // 第12行
}
```

6

编译结果:

```
1>test.cpp(12) : error C2668: "abs": 对重载函数的调用不明确
1>   test.cpp(7): 可能是 "float abs(float)"
1>   test.cpp(5): 或    "long abs(long)"
1>   c:\program files\microsoft visual studio 9.0
   \vc\include\stdlib.h(380): 或    "int abs(int)"
1>   试图匹配参数列表 "(double)"时
1>test - 1 个错误, 0 个警告
为什么呢?
```

因为-1.5是C/C++默认的double类型, 编译器找不到相应double类型的abs函数, 除非写成-1.5f

7

```
#include <iostream>
using namespace std;
float f(int i) { return (float)(i+3); }
int f(float i) { return (int)(i+2); }
void main( )
{ float f1=0;
  for (int i=0; i<5; i++)
  { f1 += f(f1));
    cout << f1 << endl;
  }
}
运行结果是什么?
```

```
5
15
35
75
155
请按任意键继续...
```

8

2.3 函数重载与缺省参数

```
#include <iostream>
using namespace std;
double f(double x)
{ return x*x;
}
double f(double x, double y=0)
{ return x*x+y*y;
}
void main( )
{ double x(3.0), y(4.0);
  cout << f(x) << endl; // 第11行
  cout << f(x,y) << endl;
}
```

9

编译结果:

```
test.cpp(11) : error C2668: "f": 对重载函数的调用不明确
1>   test.cpp(6): 可能是 "double f(double,double)"
1>   test.cpp(3): 或    "double f(double)"
1>   试图匹配参数列表 "(double)"时
1>test - 1 个错误, 0 个警告
```

原因是:

由于函数有缺省参数说明, 导致编译器根据f(x)调用无法静态绑定应该调用f(x) 还是f(x, 0.0)

10

2.3 函数重载与缺省参数

```
#include <iostream>
using namespace std;
double f(double x) { return x*x;}
double f(double x, double y) { return x*x+y*y;}
void main( )
{ double x(3.0), y(4.0);
  cout << f(x) << endl;
  cout << f(x,y) << endl;
}
```

而如上的写法, 去掉缺省参数, 编译将没有任何错误。因此在写重载函数时, 要注意避免同时使用函数缺省参数说明引起的调用二义性 (ambiguous)。

11

2.3 函数重载与缺省参数

上例也可等效修改为:

```
#include <iostream>
using namespace std;
double f(double x, double y=0.0)
{ return x*x + y*y;
}
void main( )
{ double x(3.0), y(4.0);
  cout << f(x) << endl;
  cout << f(x, y) << endl;
}
```

12

3. 类

- 类（class）是抽象数据类型的实现。
- 类是将数据和相应对这些数据的操作函数进行封装，并设置访问权限，class不同于struct、union，后者是纯数据类型，但不包括函数（操作）。
- **类是对象的抽象，而对象是类的实例。**类是抽象的，不占用内存，而对象是具体的，占用存储空间。类是用于创建对象的蓝图，它是一个定义包括在特定类型的对象中的方法和变量的软件模板（模子）。
- 类可以继承和派生，并引出了访问权限控制和调整、保护成员、成员函数名静态和动态绑定（虚函数）等一系列问题。

13

目录

- 3.1 类的定义
- 3.2 构造函数/析构函数
- 3.3 常成员函数
- 3.4 静态成员
- 3.5 友元
- 3.6 嵌套类
- 3.7 类的向前引用
- 3.8 this指针
- 3.9 类的组合问题
- 3.10 指向类的静态成员的指针
- 3.11 指向静态成员函数的函数指针
- 3.12 类与对象数组
- 3.13 类与动态对象

14

3.1 类的定义

```
class Tdate
{ public:
    void Set(int d, int m, int y);
    int IsLeapYear( );
    void Print( );
private:
    int day, month, year;
};
```

这是一个日期Tdate类的定义，其中：

15

3.1 类的定义（续1）

- Tdate类中包含成员变量day, month, year和成员函数Set、IsLeapYear、Print。
- 成员变量day, month, year是Tdate类的私有成员，仅本类成员函数Set、IsLeapYear、Print可以访问，其他外部函数不可直接访问。
- 函数Set、IsLeapYear、Print被称为成员函数，是Tdate类的公有部分，其他函数可以调用这些成员函数，通过它们间接访问Tdate类的私有成员。
- 即使是在main函数中也不能直接访问类的私有成员或私有函数，也就是说，任何类外的函数要访问类的私有成员或函数，必须通过本类的公有成员函数才能间接访问。

16

```
void Tdate::Set(int d,int m, int y)
{ day=d; month=m; year = y; }
int Tdate::IsLeapYear( )
{ return(year %4==0 && year%100!=0) || (year%400)==0);
}
int Tdate::Print( )
{ cout <<month<< "."<<day<< "."<<year<<endl; }
void main( )
{ Tdate Today, Tomorrow; //定义了两个对象
  Today.Set(25, 2, 2020);
  Tomorrow.Set(26, 2, 2020);
  if (Today. IsLeapYear( ))
    { Today. Print( ) ; cout << "Is Leap Year!";}
}
```

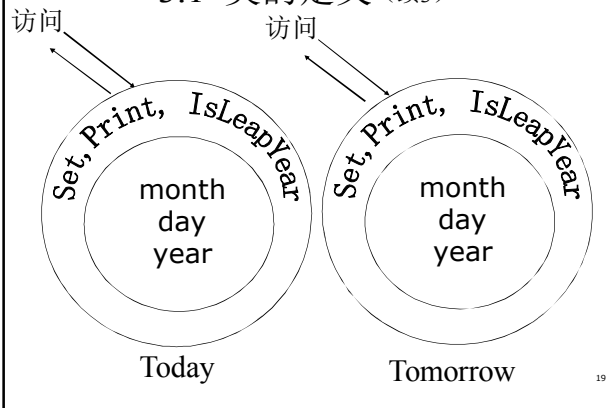
17

3.1 类的定义（续2）

- 成员函数在类外定义时要加上类的前缀Tdate::，表明是Tdate这个类的成员函数，或者说，此成员函数隶属于Tdate类。不同类的成员函数可以同名，它们之间完全互不相干，不是重载函数。
- 利用已声明的类可以定义一个或多个对象，上面的例子就用Tdate类定义了两个对象Today和Tomorrow。
- 对象之间无论是数据还是成员函数都是独立的，可以认为每个对象是一个自封闭的单元。
- 但一个对象所占内存只是存放数据成员，并不存放成员函数的副本。

18

3.1 类的定义 (续3)



```
#include <iostream>
using namespace std;
class Tdate // 可以把成员函数直接在类说明中定义, 这样
{ public: // 成员函数若符合条件, 将自动成为内联函数
    void Set(int d,int m, int y)
    { day=d; month=m; year = y; }
    int IsLeapYear( )
    { return(year%4==0 && year%100!=0) || (year%400==0); }
    void Print( )
    { cout <<month<< " "<<day<< " "<<year<<endl; }
private:
    int day, month, year;
};

int main( )
{ Tdate Today, Tomorrow; //定义了两个对象
  Today.Set(25, 2, 2020); Tomorrow.Set(26, 2, 2020);
  if(Today.IsLeapYear( ))
  { Today.Print( ); cout << "Is Leap Year!"; }
}
```

3.2 构造函数/析构函数

- ✓ 类建立后往往需要初始化,例如, 申请动态内存, 程序结束时释放动态内存, 若忘记释放会出现内存泄露。
- ✓ C++提供了构造函数/析构函数: 与类同名的成员函数被称为是构造函数(constructor)。
- ✓ 与类同名带前缀 ~ 的成员函数被称为是析构函数(destructor)。
- ✓ 构造函数/析构函数不能在程序中显式地直接被调用, 而是在对象创建和撤消时隐式地自动被调用。

3.2 构造函数/析构函数 (续1)

- ✓ 若一个类中没有定义构造函数/析构函数, 则编译器会自动生成这样的函数, 叫做公有构造函数/析构函数。
- ✓ 构造函数可以多个同名 (函数重载), 以达到各种初始化的目的。
- ✓ 同名的构造函数必须在类的public声明中逐个列出。
- ✓ 注意: 构造函数/析构函数没有返回类型。

```
#include <iostream>
using namespace std;
class Myclass {
public:
    Myclass(int i); // 构造函数
    ~Myclass( ); // 析构函数
    void Print( );
private:
    int member;
}; // 注意: 这个分号不能少!!!
Myclass::Myclass(int i)
{ member = i;
  cout << "constructor called " <<member <<endl;
}
Myclass::~Myclass( )
{ cout << "destructor called " <<member <<endl;
}
```

```
void Myclass::Print( )
{ cout << "Print called => " <<member <<endl; }
void main( )
{ Myclass Object(10); // Object的初值为10
  Object.Print( );
  Myclass Object2(20); // Object2的初值为20
  Object2.Print( );
}
```

- 编译运行会产生如下的结果:
(注意其中的析构顺序, 与构造顺序完全相反!)
- ```
constructor called 10
Print called => 10
constructor called 20
Print called => 20
destructor called 20
destructor called 10
```

## 3.2 构造函数/析构函数 (续2)

- 分析: 在定义Myclass对象时, 都赋了初值:  
Myclass Object(10); // Object的初值为10  
Myclass Object2(20); // Object2的初值为20
- 如果想定义对象而不赋初值, 必须再加一个无参数的构造函数Myclass( )。
- 现在的类说明不赋初值是不行的, 因为构造函数Myclass要求必须有初值。
- 只有对构造函数进行函数重载后, 定义Myclass对象时, 才可以赋不赋初值都可以。

25

```
#include <iostream>
using namespace std;
class Myclass {
public:
 Myclass(int i);
 Myclass();
 ~Myclass() // 析构函数
 {cout << "destructor called " << member << endl;}
 // 注意: 以上3个函数都没有返回类型
 void Print();
private: int member;
};

Myclass::Myclass(int i) // 有参数构造函数
{
 member = i;
 cout << "constructor called " << member << endl;
}

Myclass::Myclass() // 无参数构造函数
{
 member = 0; // 自动赋初值0
 cout << "constructor2 called " << member << endl;
}
```

26

```
void Myclass::Print()
{ cout << "Print called => " << member << endl; }
void main()
{ Myclass Object(10); // Object对象的初值为10
 Object.Print();
 Myclass Object2; // Object2对象不赋初值
 Object2.Print();
}
```

- 编译运行的结果为:  
constructor called 10  
Print called => 10  
constructor2 called 0  
Print called => 0  
destructor called 0  
destructor called 10

27

```
#include <iostream>
using namespace std;
class Tdate
{ public:
 Tdate(int d,int m, int y) { day=d; month=m; year=y;
 cout << "constructor called "; Print(); }
 ~Tdate(){ cout << "destructor called "; Print(); }
 void Set(int d,int m, int y) { day=d; month=m; year = y; }
 int IsLeapYear()
 { return(year%4==0 && year%100!=0) || (year%400==0); }
 void Print() { cout << month << "/" << day << "/" << year << endl; }
private:
 int day, month, year;
};

void main()
{ Tdate Today(25, 2, 2020), Tomorrow(26, 2, 2020);
 if(Today.IsLeapYear())
 { Today.Print(); cout << "is Leap Year!\n"; }
 else { Today.Print(); cout << "is not Leap Year!\n"; }
}
```

28

编译运行的结果:

```
constructor called 2/25/2020
constructor called 2/26/2020
2/25/2020
is not Leap Year!
destructor called 2/26/2020
destructor called 2/25/2020
请按任意键继续...
```

再次提醒注意:

析构顺序与构造顺序完全相反, 一一对应!

29

## 3.2 构造函数/析构函数 (续3)

也可以通过把构造函数改写成带缺省参数的函数, 实现定义对象时赋不赋初值都可以。实际上是自动赋初值!

```
class Myclass {
public:
 Myclass(int i=0);
 ~Myclass();
 void Print();
private:
 int member;
};

Myclass::Myclass(int i) //注意:这里不要再写int i=0,否则会编译出错
{
 member = i;
 cout << "constructor called " << member << endl;
}

// error C2572: "Myclass::Myclass": 重定义默认参数: 参数1
```

30

### 3.2.1 内联构造函数 (1)

1. 可以通过把构造函数前加inline,使之成为内联构造函数

```
class MyClass {
public:
 MyClass(int i=0);
 ~MyClass();
 void Print();
private:
 int member;
};
inline MyClass::MyClass(int i)
{
 member = i;
 cout << "constructor called " << member << endl;
}
```

31

### 3.2.1 内联构造函数 (2)

2. 可以通过说明时把构造函数体直接放在类内,使之成为内联构造函数。类的构造函数体或成员函数体放在类内,则这个函数将自动成为内联函数(当然前提是:此函数符合内联函数的限制条件)。

```
class MyClass {
public:
 MyClass(int i=0) // 带默认缺省值的内联构造函数
 {
 member = i;
 cout << "constructor called " << member << endl;
 }
 ~MyClass();
 void Print();
private:
 int member;
};
目的只有一个: 提高执行效率。
```

32

### 3.2.2 复制构造函数

- 复制(拷贝)构造函数是一种特殊的构造函数,其形参只有一个且是本类对象的引用。
- 作用是:使用一个已经存在的对象(由复制构造函数的参数指定),去构造并初始化同类的一个新对象。要点是**构造(生成)一个新对象!**

```
class A {
public:
 A(形参表); // 构造函数
 A(A &对象名); // 复制构造函数
private:
 // 成员
};
A::A(A &对象名) //复制构造函数的实现
{
 函数体 //一般语句格式: 成员=对象名.成员
}
```

33

### 3.2.2 复制构造函数

**复制构造函数**在以下三种情况下会被自动调用:

- (1) 当用类的一个对象去初始化该类的另一个新定义的对象(用对象赋初值);
- (2) 如果函数的形参是类的对象,调用函数时,进行形参和实参结合;
- (3) 如果函数的返回值是类的对象,函数执行完成返回对象给调用者。

**注意:** 1. 如果函数的形参或者函数的返回值是类的对象的引用,这只是传递对象的地址而不需要传递对象,则不会调用相应复制构造函数。

2. 两个对象之间的赋值不调用相应复制构造函数,因为只是赋值没有构造过程。

34

### 3.2.2 复制构造函数

```
#include <iostream>
using namespace std;
class Point { // Point 点类的声明
public: // 外部接口
 Point(int xx=0,int yy=0) // 构造函数,带缺省值
 {
 X=xx; Y=yy;
 }
 Point(Point &p); // 复制构造函数
 int GetX() {return X;}
 int GetY() {return Y;}
private: // 私有成员
 int X,Y;
};
```

35

### 3.2.2 复制构造函数

```
Point::Point(Point &p) //复制构造函数形参是对象的引用
{
 static int n=0; // 静态计数器,统计函数被调用次数
 X=p.X; Y=p.Y; //注意:可直接访问另一个对象的私有成员
 cout << "复制构造函数第" << ++n
 << "次被调用X=" << X << " Y=" << Y << endl;
}
void fun1(Point p) // 函数形参为Point类对象的函数
{
 cout << p.GetX() << endl;
}
Point fun2() //函数的返回值为Point类对象的函数
{
 Point A(1,2);
 return A;
}
```

36

```

void main()
{
 Point A(4,5); // 第一个对象A
 Point B(A); // 情况1, 用A初始化B。第1次调用复制构造函数
 cout <<B.GetX() <<endl;
 fun1(B); // 情况2, 对象B为fun1的实参,
 // 传递参数时, 第2次调用复制构造函数
 B=fun2(); // 情况3, 函数fun2的返回值是类对象,
 // 函数返回对象时, 第3次调用复制构造函数
 cout <<B.GetX() <<endl;
 B=A; // 赋值语句并不调用复制构造函数, 因为没有构造过程
}

```

运行结果:  
 复制构造函数第1次被调用X=4 Y=5  
 4  
 复制构造函数第2次被调用X=4 Y=5  
 4  
 复制构造函数第3次被调用X=1 Y=2  
 1

37

### 3.2.3 浅复制与深复制

在C语言中我们讲过, struct结构体可以整体赋值,

例如: struct student {

```

 char name[10];
 int sno;
 double grade;
} a={"ZhangSan", 2019011001, 85.5}, b;
b = a; // a的各成员都对应赋值给b

```

即结构整体复制, 将一个结构变量的各成员的值对应赋值给另一个结构变量的各成员:

整型、实型、字符串、指针。

但指针赋值会使得不同的结构体变量中的成员指针指向同一块内存。

38

### 3.2.3 浅复制与深复制(2)

如果student结构体改为:

```

struct student {
 char *name;
 int sno;
 double grade;
} a, b;
a.name = (char *)malloc(sizeof(char)*10);
strcpy(a.name, "ZhangSan");
a.sno=2019011001; a.grade=85.5;
b = a; // a的各成员都对应赋值给b的各成员

```

则a.name指针和b.name指针指向同一块内存。因此结构体整体赋值属于浅复制 (shallow copy)。

39

### 3.2.3 浅复制与深复制(3)

例如, 有这样一个字符串类的程序

```

#include <iostream>
#include <cstring>
using namespace std;
class Strings {
public:
 Strings(const char *s);
 ~Strings();
 void Print();
 void Set(const char *s);
private:
 int length;
 char *str;
};

```

40

```

Strings::Strings(const char *s)
{
 length = strlen(s);
 str = new char[length+1];
 strcpy(str, s);
 cout << "构造函数被调用! " << endl;
}
Strings::~Strings()
{
 delete []str;
 cout << "析构函数被调用! " << endl;
}
void Strings::Print()
{
 cout << str << endl;
 cout << "length=" << length << endl;
}

```

41

```

void Strings::Set(const char *s)
{
 length = strlen(s);
 delete []str;
 str = new char[length+1];
 strcpy(str, s);
}
void main()
{
 Strings S1("Hello!");
 Strings S2(S1);
 S1.Print();
 S2.Print();
 S1.Set("New String!");
 S1.Print();
 S2.Print();
}

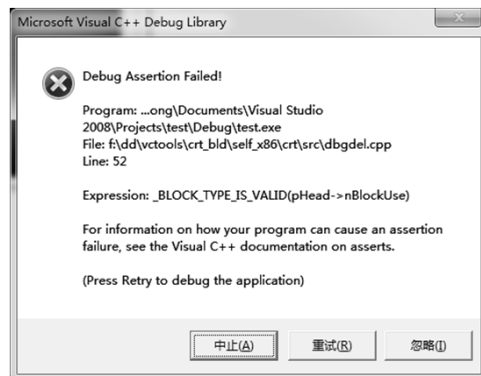
```

42

运行结果是：  
构造函数被调用！  
Hello!  
length=6  
Hello!  
length=6  
New String!  
length=11  
New String!  
length=6  
析构函数被调用！

出现如下错误信息！或死机不动了。为什么？？？

43



44

### 3.2.3 浅复制与深复制(4)

例2, 把上例中的char \*str; 改为: char str[100];  
#include <iostream>  
#include <cstring>  
using namespace std;  
class Strings {  
public:  
    Strings(const char \*s);  
    ~Strings();  
    void Print();  
    void Set(const char \*s);  
private:  
    int length;  
    char str[100];  
};

45

```
Strings::Strings(const char *s)
{ length = strlen(s);
 if (length < 100)
 strcpy(str, s);
 cout << "构造函数被调用！" << endl;
}
Strings::~Strings()
{ cout << "析构函数被调用！" << endl;
}
void Strings::Print()
{ cout << str << endl;
 cout << "length=" << length << endl;
}
```

46

```
void Strings::Set(const char *s)
{ length = strlen(s);
 if (length < 100)
 strcpy(str, s);
}
void main()
{
 Strings S1("Hello!");
 Strings S2(S1);
 S1.Print();
 S2.Print();
 S1.Set("New String!");
 S1.Print();
 S2.Print();
}
```

47

运行结果是：  
构造函数被调用！  
Hello!  
length=6  
Hello!  
length=6  
New String!  
length=11  
Hello!  
length=6  
析构函数被调用！  
析构函数被调用！  
请按任意键继续...  
程序正常结束！！！！！！ 为什么？？？

48



### 3.2.4 深复制

如果在上例中类的定义中增加一个复制构造函数:

```
Strings::Strings(Strings &p)
{ length = strlen(p.str);
 str = new char[length+1];
 strcpy(str, p.str);
 cout << "复制构造函数被调用！" << endl;
}
```

不再是简单的对象指针赋值，而是为新对象中的指针变量申请新的内存，并复制字符串。这就是所谓的：**深复制**（deep copy）

上面的例子的程序变为：

49

```
#include <iostream>
#include <cstring>
using namespace std;
class Strings {
public:
 Strings(const char *s);
 Strings(Strings &p);
 ~Strings();
 void Print();
 void Set(const char *s);
private:
 int length;
 char *str;
};
Strings::~Strings()
{ delete []str;
 cout << "析构函数被调用！" << endl;
}
```

50

```
Strings::Strings(const char *s)
{ length = strlen(s);
 str = new char[length+1];
 strcpy(str, s);
 cout << "构造函数被调用！" << endl;
}
Strings::Strings(Strings &p)
{ length = strlen(p.str);
 str = new char[length+1];
 strcpy(str, p.str);
 cout << "复制构造函数被调用！" << endl;
}
void Strings::Print()
{ cout << str << endl;
 cout << "length=" << length << endl;
}
```

51

```
void Strings::Set(const char *s)
{ length = strlen(s);
 delete []str;
 str = new char[length+1];
 strcpy(str, s);
}
void main()
{ Strings S1("Hello!");
 Strings S2(S1);
 S1.Print();
 S2.Print();
 S1.Set("New String!");
 S1.Print();
 S2.Print();
}
```

运行结果是：

52

构造函数被调用！  
复制构造函数被调用！  
Hello!  
length=6  
Hello!  
length=6  
New String!  
length=11  
Hello!  
length=6  
析构函数被调用！  
析构函数被调用！  
请按任意键继续...



53

注意：在声明一个类时，如果不写复制构造函数，编译器会自动为此类生成一个复制构造函数，但这种自动生成的复制构造函数一般都是**浅复制**，只是把一个对象中的各个成员一一对应赋值给另一个对象的各个成员，包括指针成员的值。这会使得两个对象的指针成员指向同一块动态内存，往往会造成析构对象执行析构函数释放动态内存时，产生致命错误。

54

### 3.3 常成员函数

前面的例子Strings类中有Print成员函数：

```
void Strings::Print()
{ cout << str << endl;
 cout << "length=" << length << endl;
}
```

此函数只打印出Strings类的成员的值，并没有做任何修改，此成员函数可以定义为常成员函数：

```
void Strings::Print() const
{ cout << str << endl;
 cout << "length=" << length << endl;
}
```

即在函数名( )后加上修饰符const，可确保此函数中不会有任何修改Strings类的成员的行为，否则编译时就会出现错误信息。

例如：

55

```
#include <iostream>
#include <cstring>
using namespace std;
class Strings {
public:
 Strings(const char *s);
 Strings(Strings &p);
 ~Strings();
 void Print() const; // 常成员函数
 void Set(const char *s);
private:
 int length;
 char *str;
};
Strings::~Strings()
{ delete []str;
 cout << "析构函数被调用！" << endl;
}
```

56

```
Strings::Strings(const char *s)
{ length = strlen(s);
 str = new char[length+1];
 strcpy(str, s);
 cout << "构造函数被调用！" << endl;
}
Strings::Strings(Strings &p)
{ length = strlen(p.str);
 str = new char[length+1];
 strcpy(str, p.str);
 cout << "复制构造函数被调用！" << endl;
}
void Strings::Print() const
{ cout << str << endl;
 cout << "length=" << length++ << endl; // 第33行
} //这里试图修改length
```

57

```
void Strings::Set(const char *s)
{ length = strlen(s);
 delete []str;
 str = new char[length+1];
 strcpy(str, s);
}
void main()
{ Strings S1("Hello!");
 Strings S2(S1);
 S1.Print();
 S2.Print();
 S1.Set("New String!");
 S1.Print();
 S2.Print();
}
编译结果：
test.cpp(33) : error C2166: 左值指定const 对象
```

58

#### 3.3.1 mutable关键字

◆ 类的常成员函数中不能有任何修改类的成员值的行为，但某些情况下，能否网开一面，可以对某些特定成员做修改呢？比如计数器。

◆ 方法是：在特定成员定义时，前面加上mutable，这样的成员在常成员函数中可以被修改。例如：

```
class Strings {
public:
 Strings(const char *s);
 Strings(Strings &p);
 ~Strings();
 void Print() const;
 void Set(const char *s);
private:
 int length;
 char *str;
 mutable int Count; // 计数器，统计被打印次数
};
```

59

```
#include <iostream>
#include <cstring>
using namespace std;
class Strings {
public:
 Strings(const char *s);
 Strings(Strings &p);
 ~Strings();
 void Print() const; // 常成员函数
 void Set(const char *s);
private:
 int length;
 char *str;
 mutable int Count;
};
Strings::~Strings()
{ delete []str;
 cout << "析构函数被调用！" << endl;
}
```

60

```

Strings::Strings(const char *s)
{ length = strlen(s); Count=0;
 str = new char[length+1];
 strcpy(str, s);
 cout << "构造函数被调用！" << endl;
}
Strings::Strings(Strings &p)
{ length = strlen(p.str); Count = p.Count;
 str = new char[length+1];
 strcpy(str, p.str);
 cout << "复制构造函数被调用！" << endl;
}
void Strings::Print() const
{ cout << str << endl;
 cout<<"length="<<length<<" C="<< Count++ << endl;
} //在常函数中允许修改Count的值，因为Count用mutable说明过

```

61

```

void Strings::Set(const char *s)
{ length = strlen(s);
 delete []str;
 str = new char[length+1];
 strcpy(str, s);
}
void main()
{ Strings S1("Hello!");
 Strings S2(S1);
 S1.Print();
 S2.Print();
 S1.Set("New String!");
 S1.Print();
 S2.Print();
}
编译没有错误，运行结果：

```

```

构造函数被调用！
复制构造函数被调用！
Hello!
length=6 C=0
Hello!
length=6 C=0
New String!
length=11 C=1
Hello!
length=6 C=1
析构函数被调用！
析构函数被调用！
请按任意键继续...

```

62

### 第3次作业：

见网络学堂第3次作业

2题必做，1题选做

63